PRODI S1 SAINS DATA
FAKULTAS MATEMATIKA DAN
ILMU PENGETAHUAN ALAM

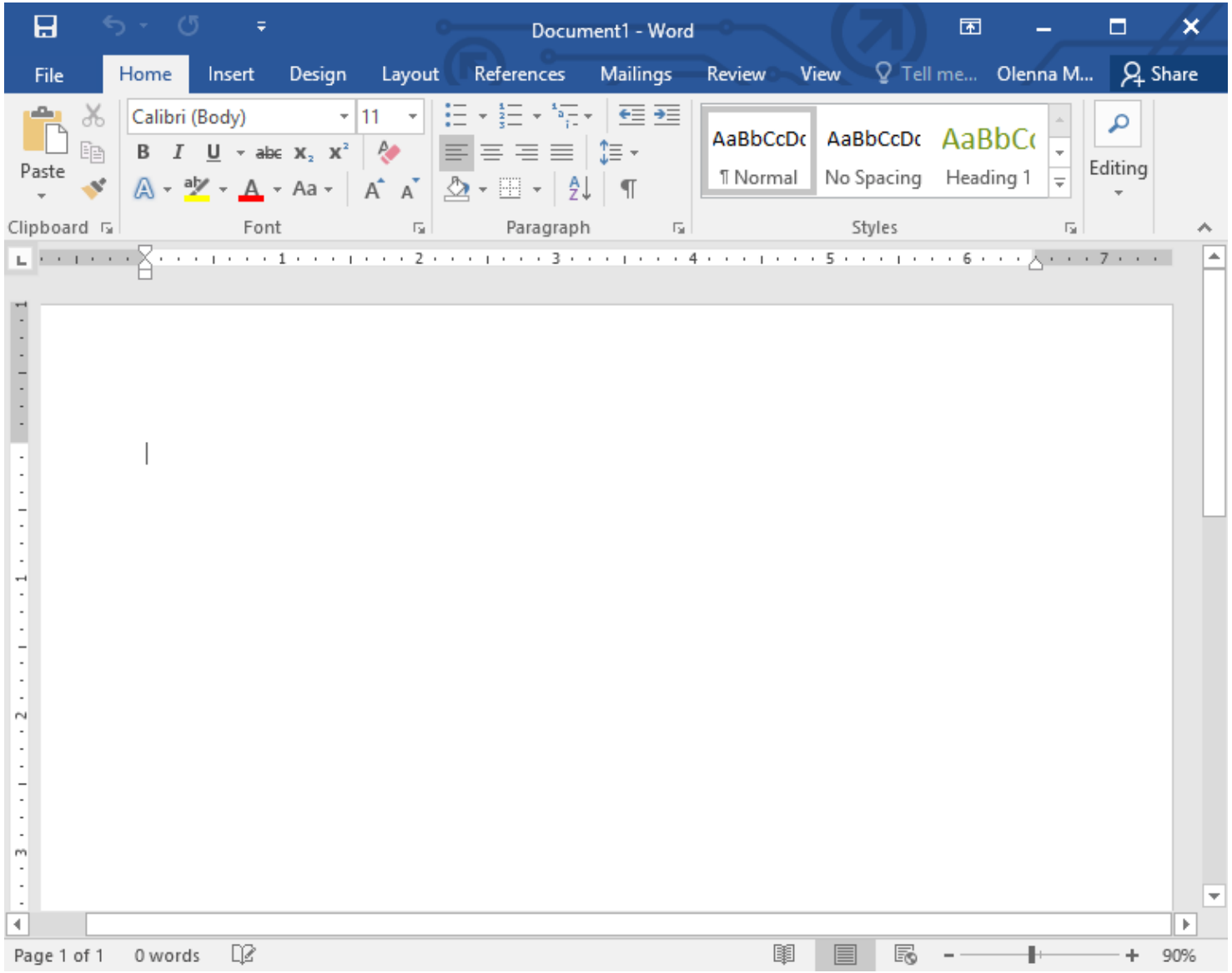
Pemrograman Dasar

Graphical User Interface (GUI)

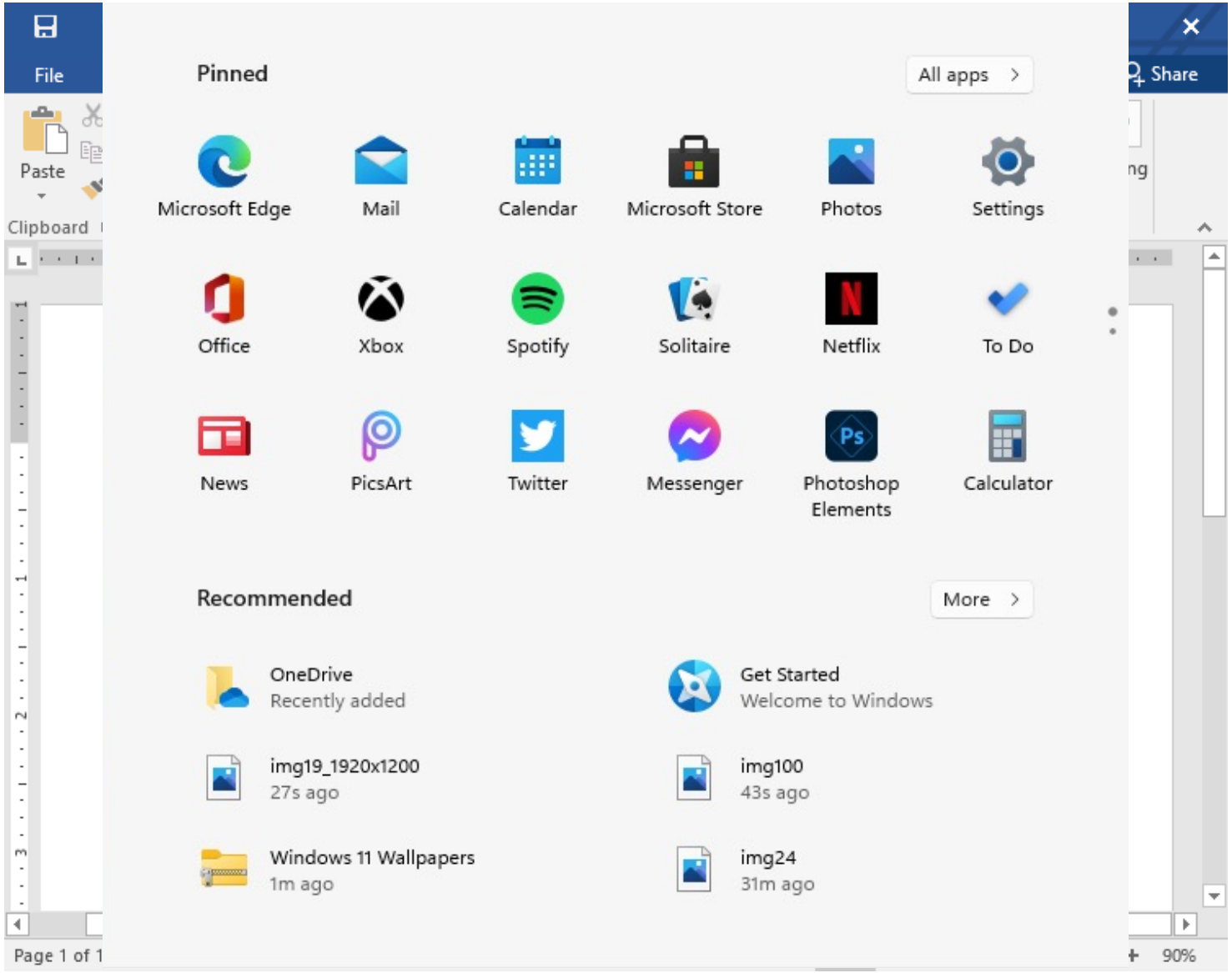
Dr. Atik Wintarti, S.Pd., M.Kom
Dr. Elly Matul Imah, S.Si., M.Kom
Riskyana Dewi Intan P, S.Si., M.Kom

2022

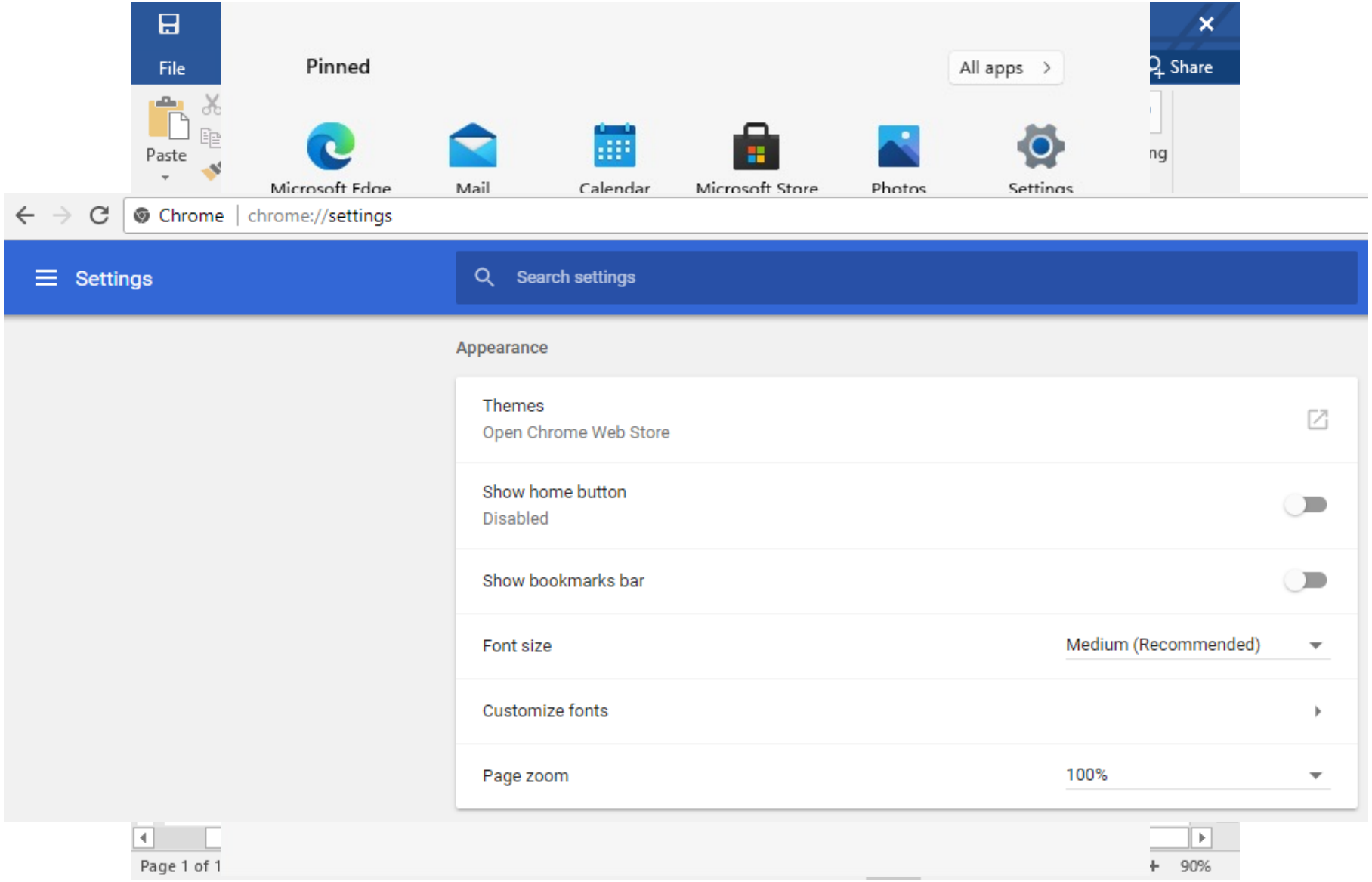
Graphical User Interface (GUI)??



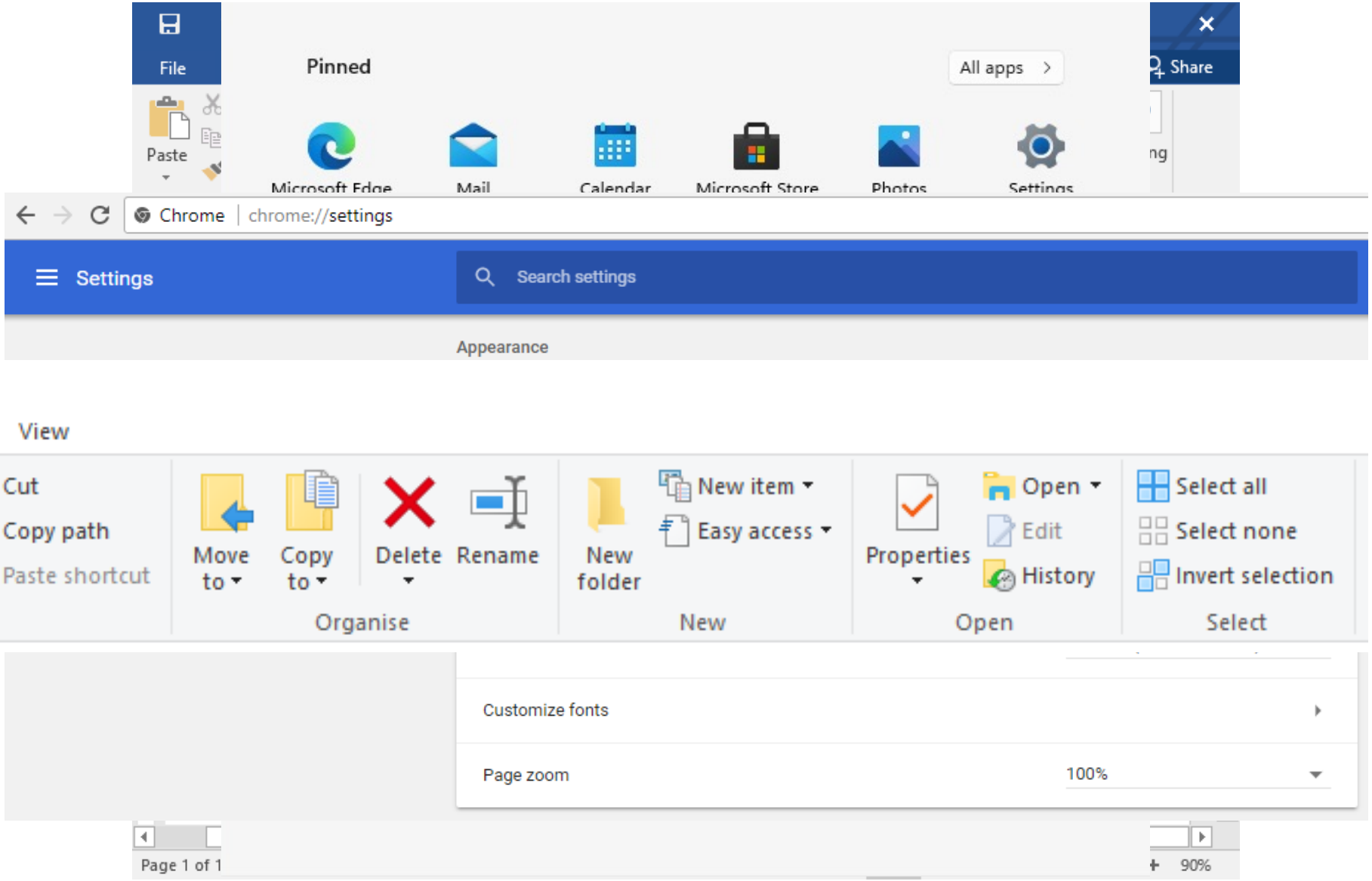
Graphical User Interface (GUI)??



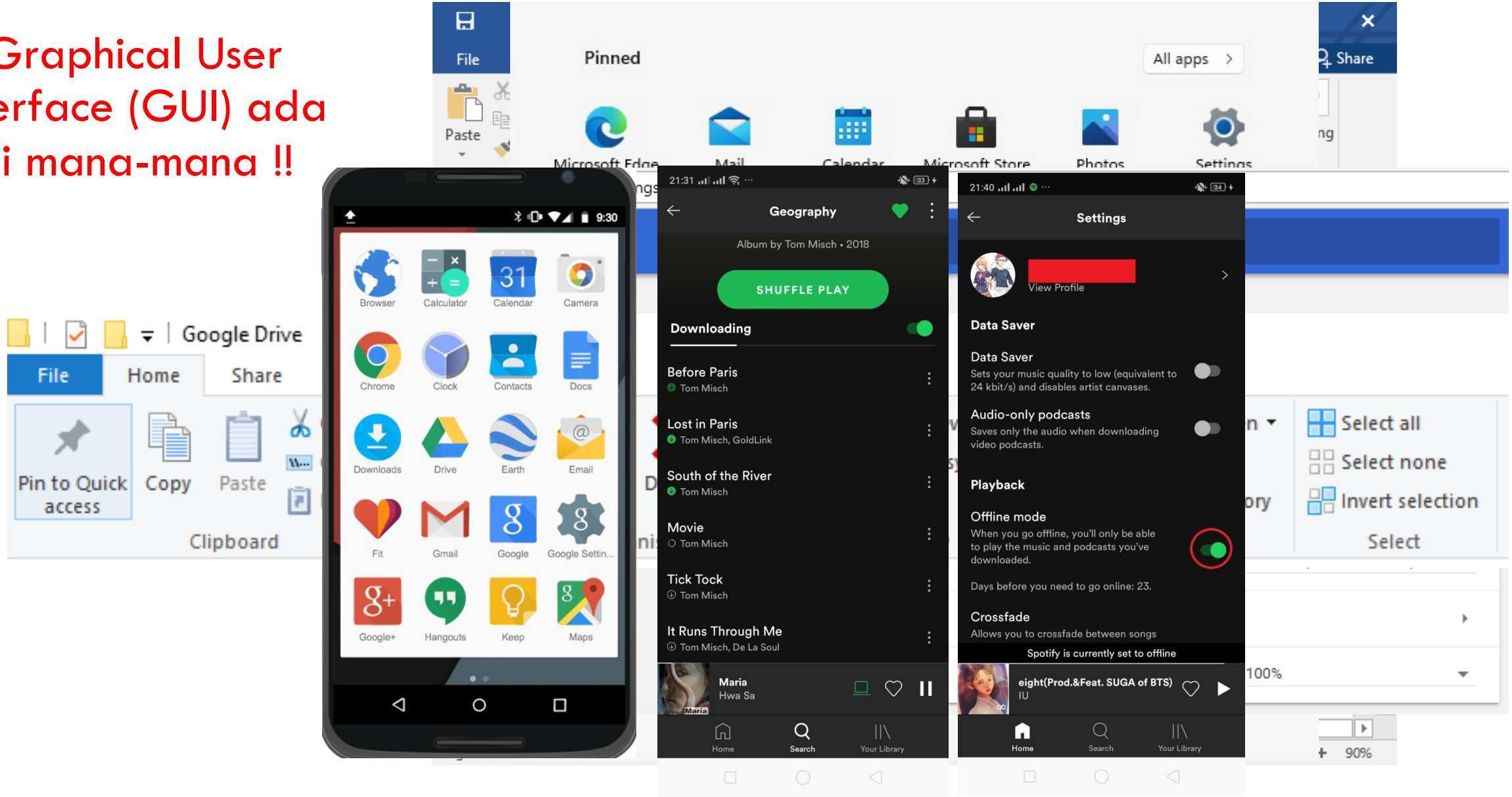
Graphical User Interface (GUI)??



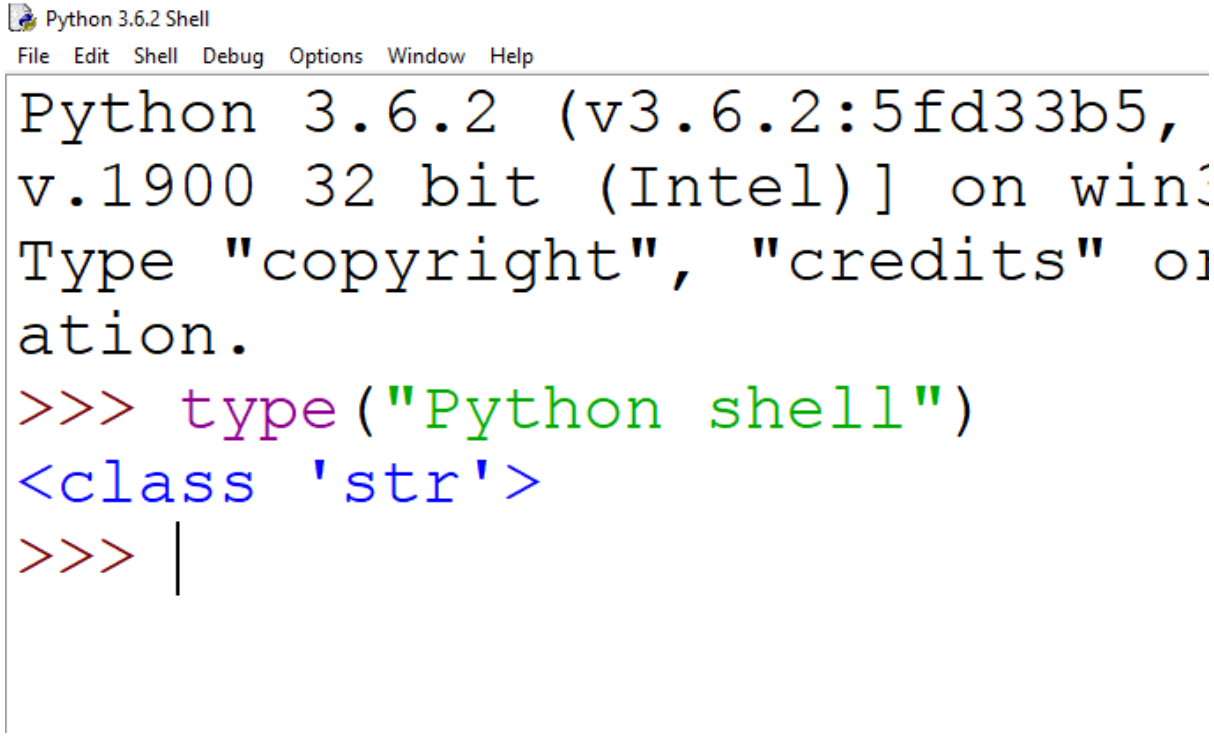
Graphical User Interface (GUI)??



Graphical User Interface (GUI) ada di mana-mana !!

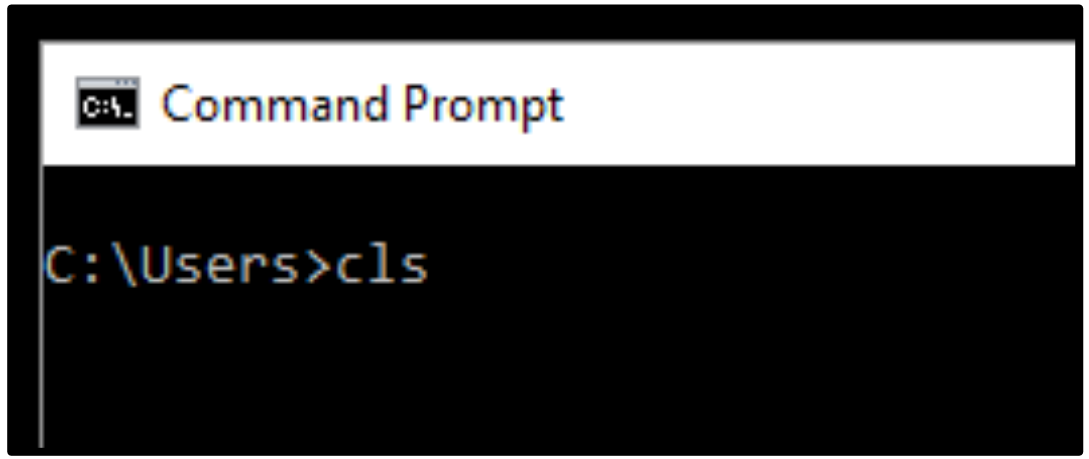


Kenapa tidak pakai Command-Line Interface (CLI)?



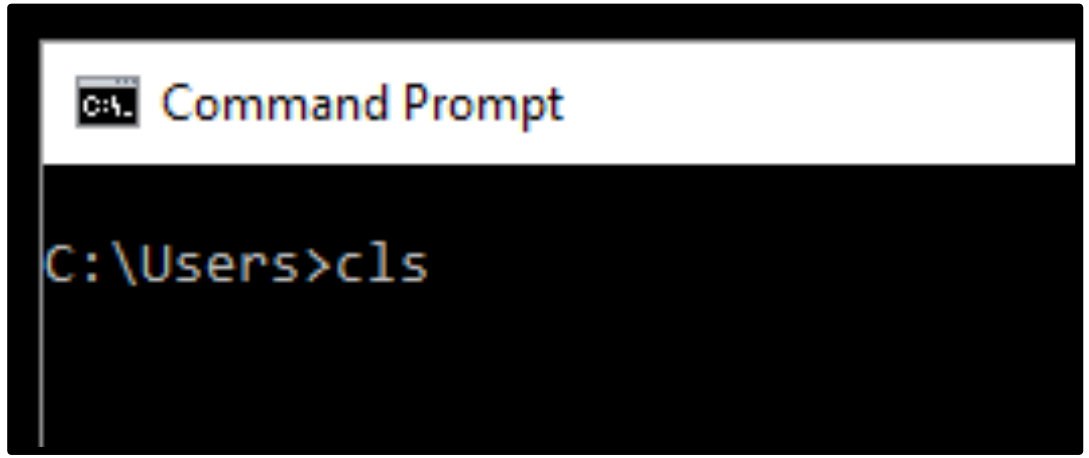
Python Shell

Kenapa tidak pakai Command-Line Interface (CLI)?



Windows Command Prompt

Kenapa tidak pakai Command-Line Interface (CLI)?



Tidak semua pengguna komputer nyaman menggunakan CLI, bayangkan kakek-nenek kamu menggunakan komputer!

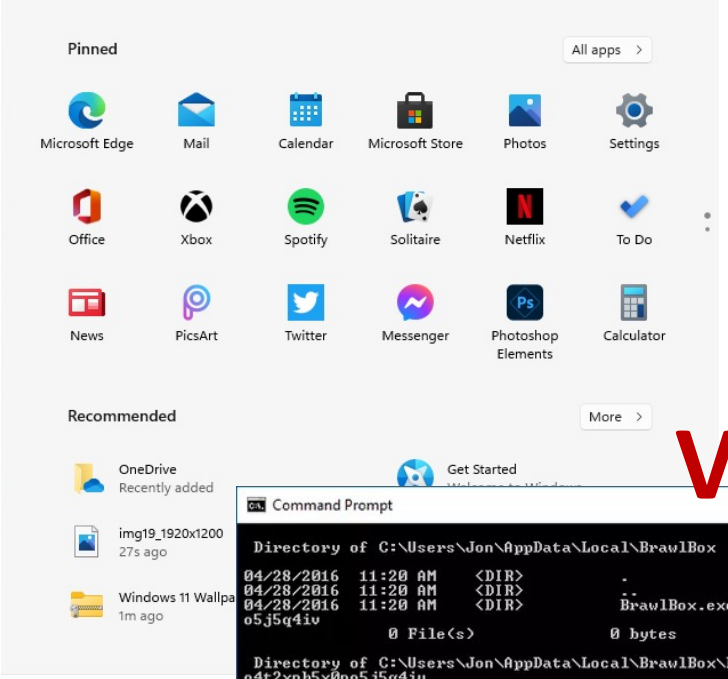
Semua ada positif dan negatifnya..

When to use GUI

- Reduce mental work
- Make results visible
- Make the barrier of the entry lower

When to use CLI

- Do things at scale: A simple CLI command can easily adjust configurations for a large group of systems
- Something needs to be scripted and automated
- For less memory usage
- ~~To look cooler (aka. hacker-style)~~



VS

Apa itu GUI?

- Suatu antarmuka (user interface) dimana pengguna berinteraksi melalui objek-objek visual (widgets), seperti tombol, checkbox, menu, dsb.
- Dengan kata lain, **visual-based interface** (kalau CLI, lebih bersifat text-based interface).

Outline

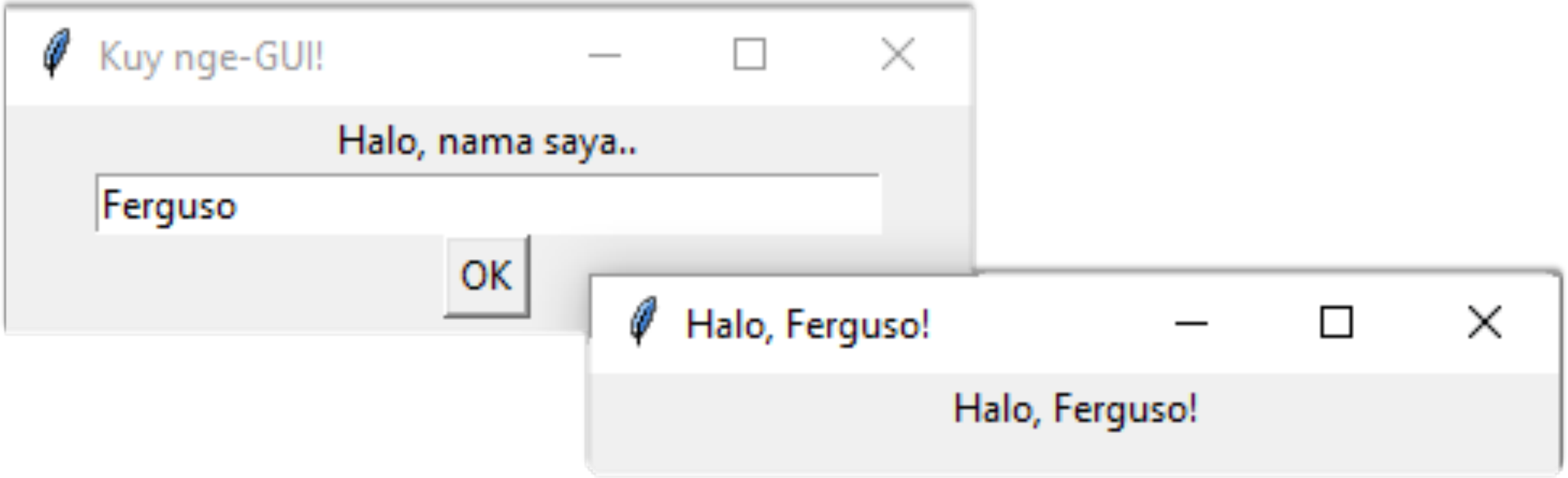
01
TKINTER

02
PyQT

tkinter: Modul GUI untuk Python

- GUI library (= modul) standar bawaan Python
- tkinter = **T**ool**K**it **INTER**face
- Cross-platform = jalan di Windows, Linux, Mac OS
- Menyediakan berbagai elemen-elemen GUI (widgets) seperti tombol, menu, entry (= text field, untuk buat form), dsb
- Event-driven = GUI akan merespons ke event-event (aksi-aksi) pengguna GUI (misalnya klik, ketik, scroll, dsb)

Tampilan GUI tkinter



Live Coding: Membuat window

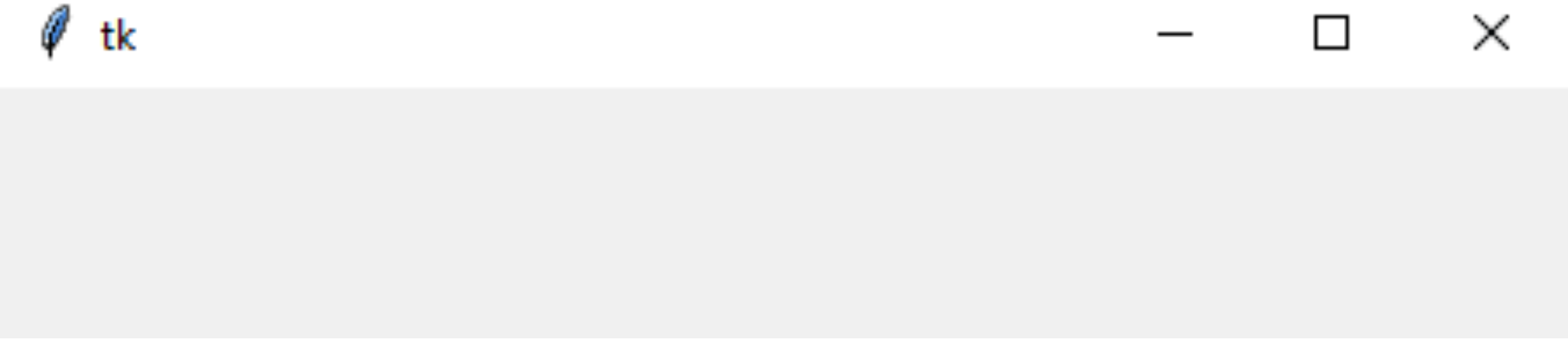
```
import tkinter
window = tkinter.Tk() #membuat window
window.mainloop() #jalankan tkinter
```

Live Coding: Membuat window

```
from tkinter import Tk #dengan cara ini jadi lebih ringkas
window = Tk() #membuat window
window.mainloop() #jalankan tkinter
```

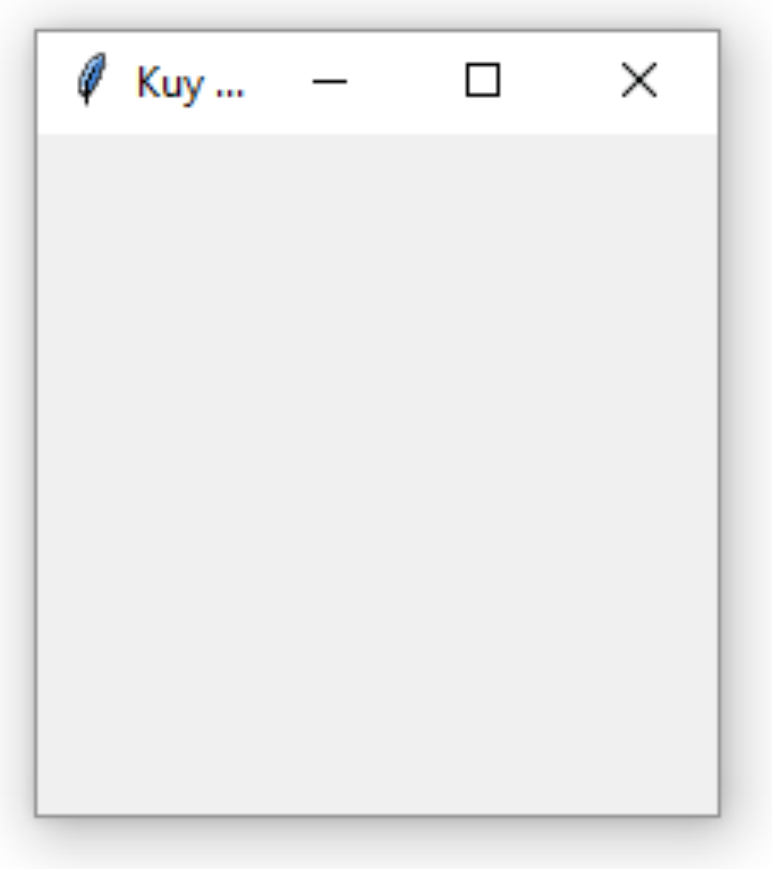
Live Coding: Membuat window

```
from tkinter import Tk #dengan cara ini jadi lebih ringkas
window = Tk() #membuat window
window.mainloop() #jalankan tkinter
```



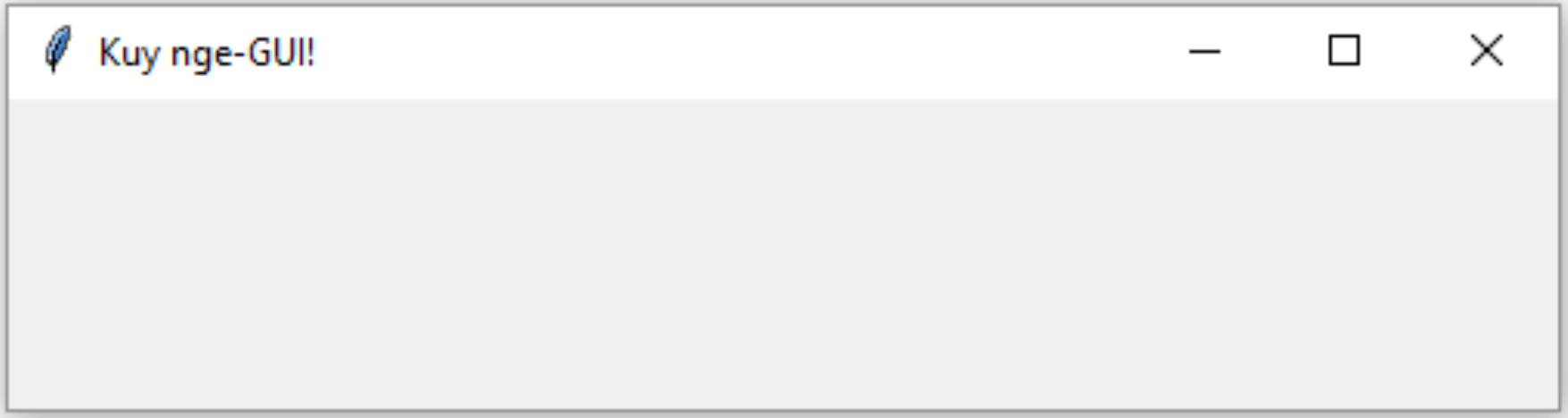
Live Coding: Membuat window dengan judul

```
from tkinter import Tk
window = Tk()
window.title("Kuy nge-GUI!")
window.mainloop()
```



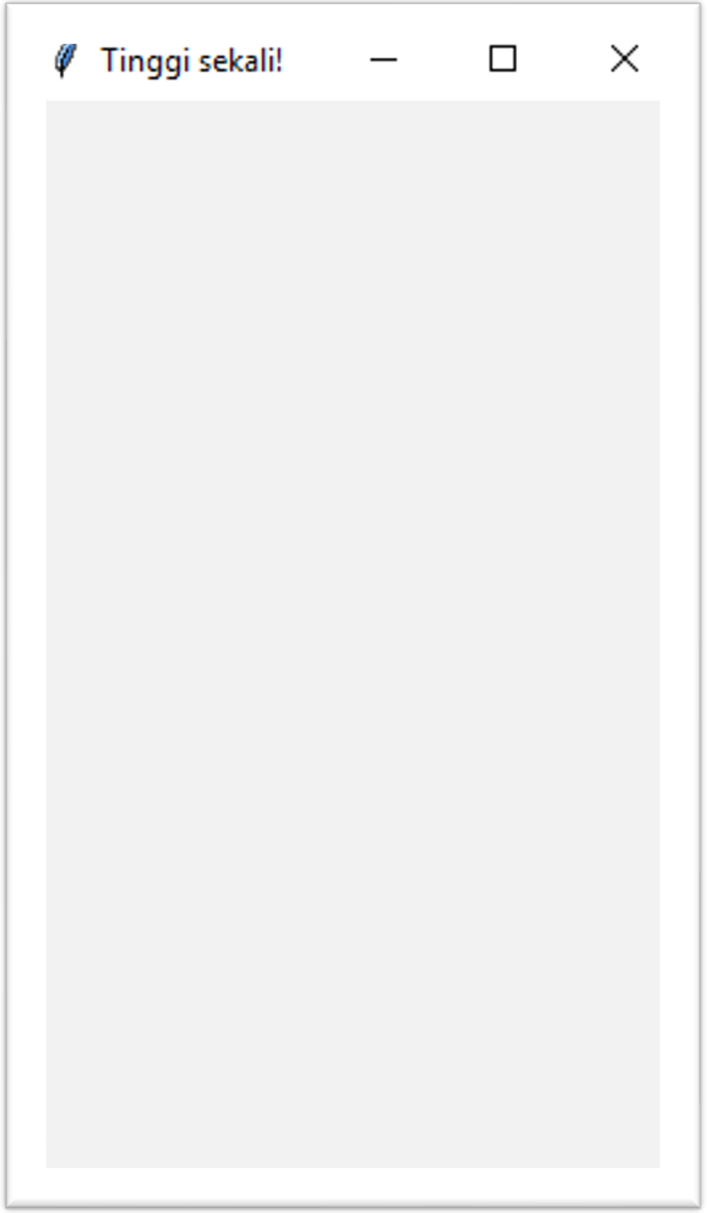
Live Coding: Atur Ukuran Window

```
from tkinter import Tk
window = Tk()
window.title("Kuy nge-GUI!")
window.geometry("500x100") # atur lebar 500, tinggi 100 pixels
window.mainloop()
```



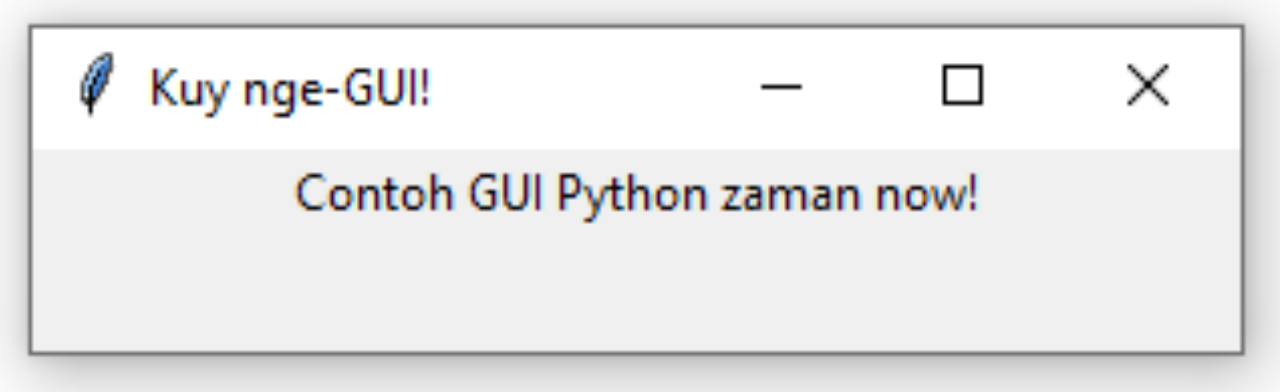
Live Coding: Atur Ukuran Window

```
from tkinter import Tk

window = Tk()
window.title("Tinggi sekali!")
window.geometry("250x5000")
window.mainloop()
```



Live Coding: Atur Ukuran Window

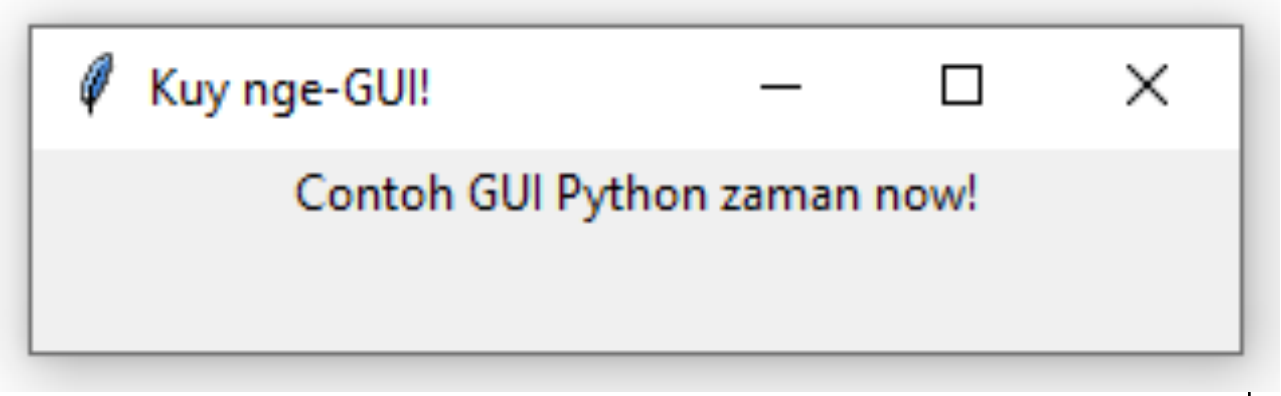
```
from tkinter import Tk

window = Tk()
window.title("Tinggi sekali!")
window.geometry("250x5000")
window.mainloop()
```



Live Coding: Menyisipkan Label

```
from tkinter import *

class MyFirstGUI:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-GUI!")
        master.geometry("300x50")

        # Label adalah salah satu widget di tkinter
        # dan argumen pertama di widget constructor selalu parent container
        # yang akan menampung widget terkait
        self.label = Label(master, text="Contoh GUI Python zaman now!")
        self.label.pack() # tempatkan labelnya di container

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```

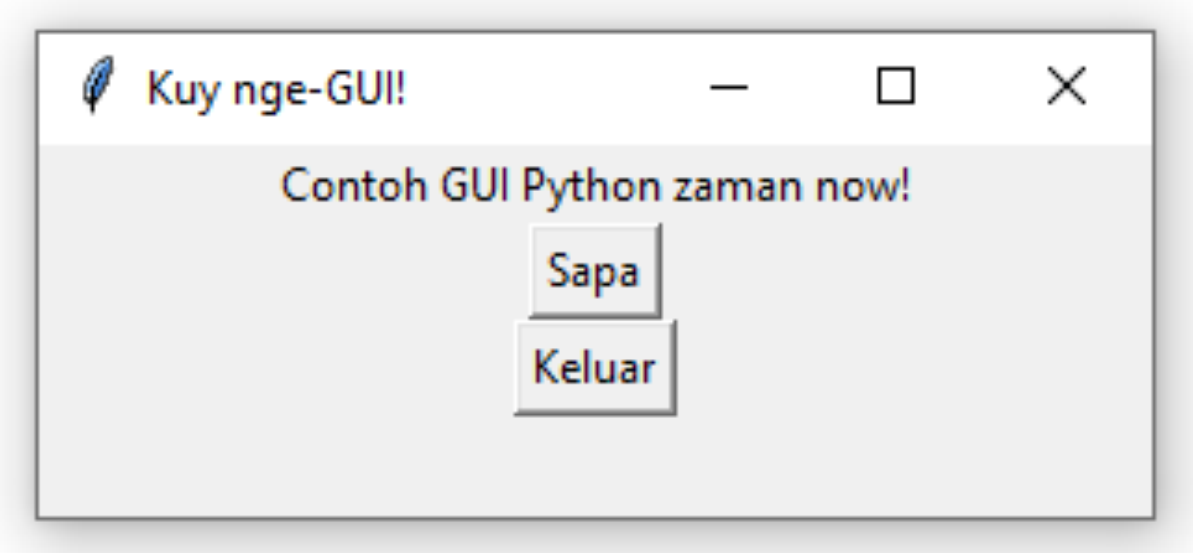
Live Coding: Menyisipkan Label




```
from tkinter import *

class MyFirstGUI:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-GUI!")
        master.geometry("300x50")

        # Label adalah salah satu widget di tkinter
        # dan argumen pertama di widget constructor selalu parent container
        # yang akan menampung widget terkait
        self.label = Label(master, text="Contoh GUI Python zaman now!")
        self.label.pack() # tempatkan labelnya di container

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```

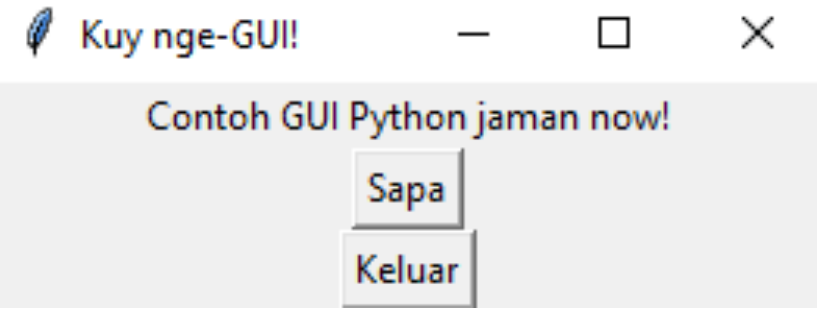
Live Coding: Menyisipkan Label dan Button

```
# di dalam __init__ kode sebelumnya, tambahkan berikut..
# jangan lupa window height di-set ke 100

self.greet_button = Button(master, text="Sapa")
self.greet_button.pack()
self.close_button = Button(master, text="Keluar")
self.close_button.pack()
```



Live Coding: Button bukan hanya hiasan

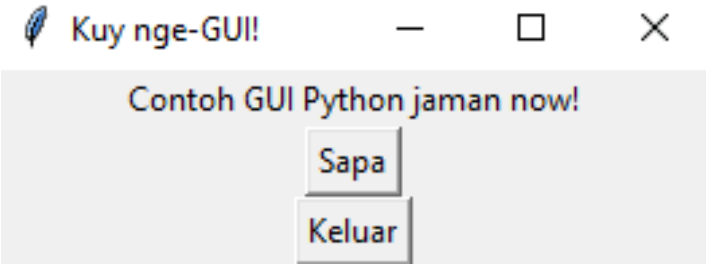
```
# ubah bagian close_button menjadi berikut

self.close_button = Button(master, text="Keluar", command=master.destroy)
```



klik button Keluar → ***Window lenyap!***

Live Coding: Button bukan hanya hiasan

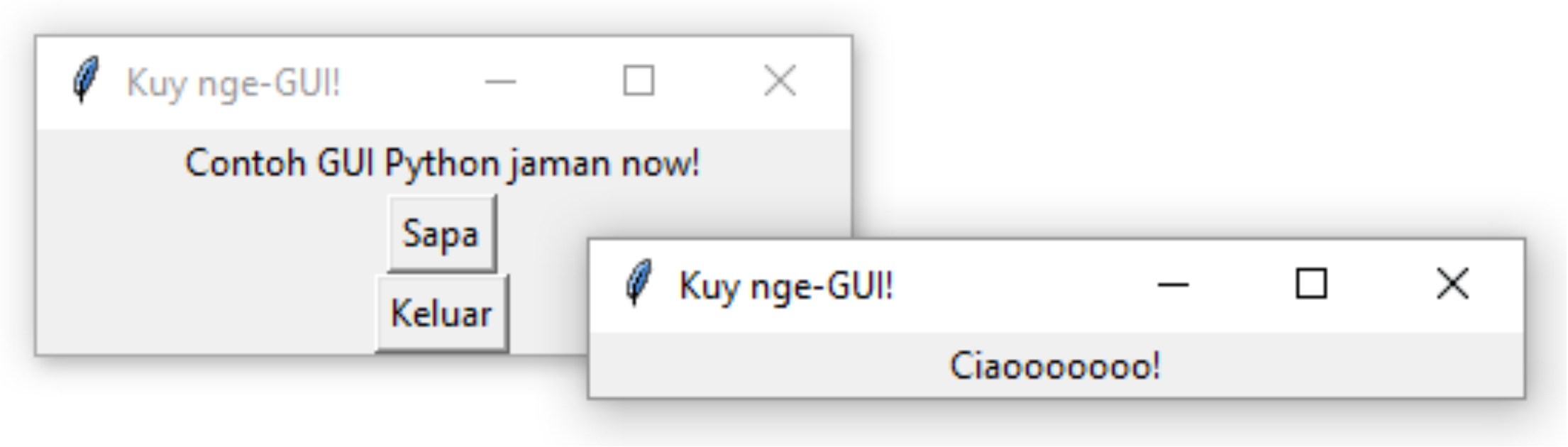
```
# di definisi class MyFirstGUI tambahkan method berikut

def sapa(self):
        print("Ciaooooo!") # sapa bahasa Italia

# ubah bagian greet_button menjadi berikut

self.greet_button = Button(master, text="Sapa", command=self.sapa)
```



klik button Sapa → ***Cetak "Ciaooooo!" pada command line/shell***

Live Coding: Button bukan hanya hiasan (Pop-Up)

```
# di definisi class MyFirstGUI tambahkan method berikut

def sapa(self):
    window = Toplevel(self.master)
    Label(window, text="Ciaoooooooo!").pack()
```

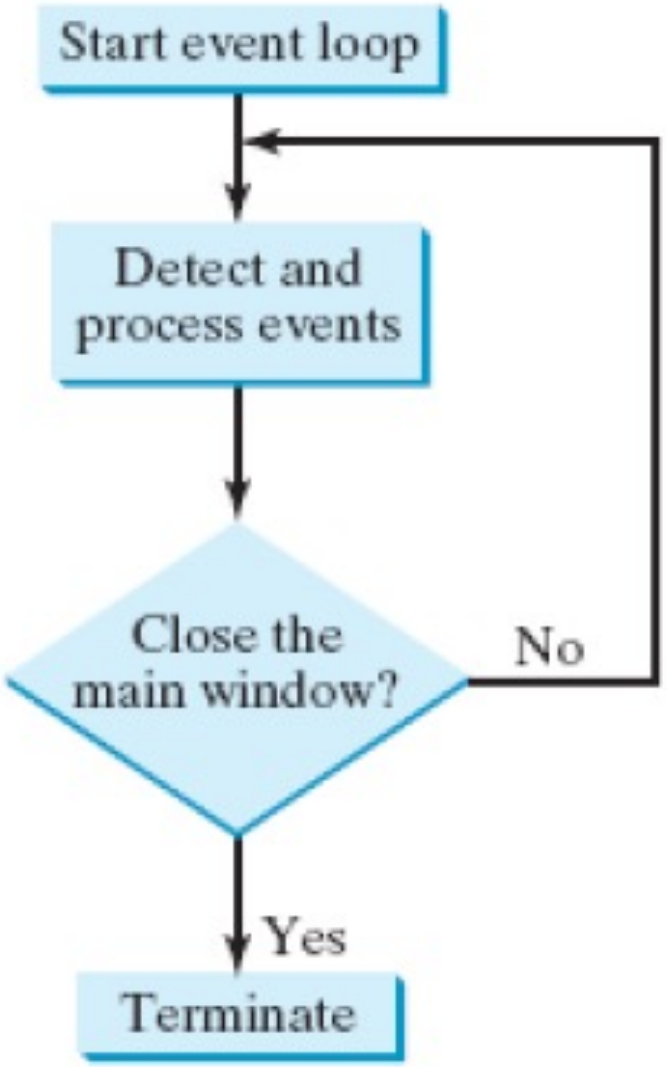


Live Coding: Button bukan hanya hiasan (Ubah Warna)

```
# ubah bagian greet_button dan close_button menjadi berikut

self.greet_button = Button(master, text="Sapa", command=self.sapa, fg="green")
self.close_button = Button(master, text="Keluar", command=master.destroy, bg="red")
```

Event Processing

- tkinter GUI bersifat event-driven,
  yakni selalu menunggu aksi (= event) pengguna
- Dispesifikasikan dengan method **mainloop()**
- Method tersebut membuat event-loop,
  yang akan memproses event terus menerus
  sampai window-nya ditutup (atau ada error!)

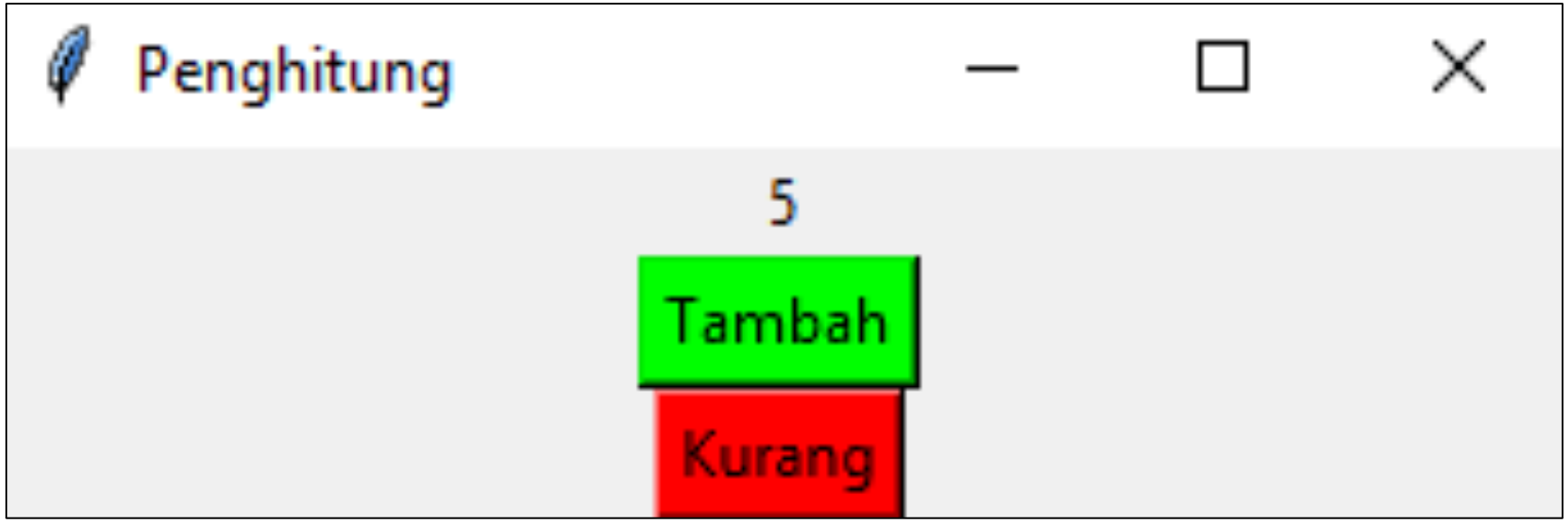


Event Processing

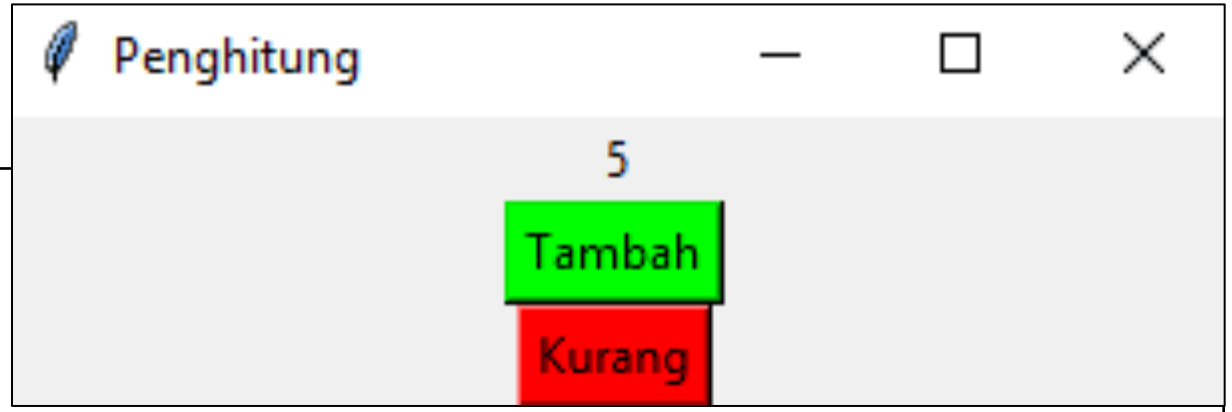
- Widget dapat diberi event handler (= callback function), yang dieksekusi apabila terjadi suatu event pada widget
- Misalnya pada saat pengguna klik suatu button:

```
self.greet_button = Button(master, text="Sapa", command=self.sapa)
self.close_button = Button(master, text="Keluar", command=master.destroy)
```

Live Coding : Penghitung

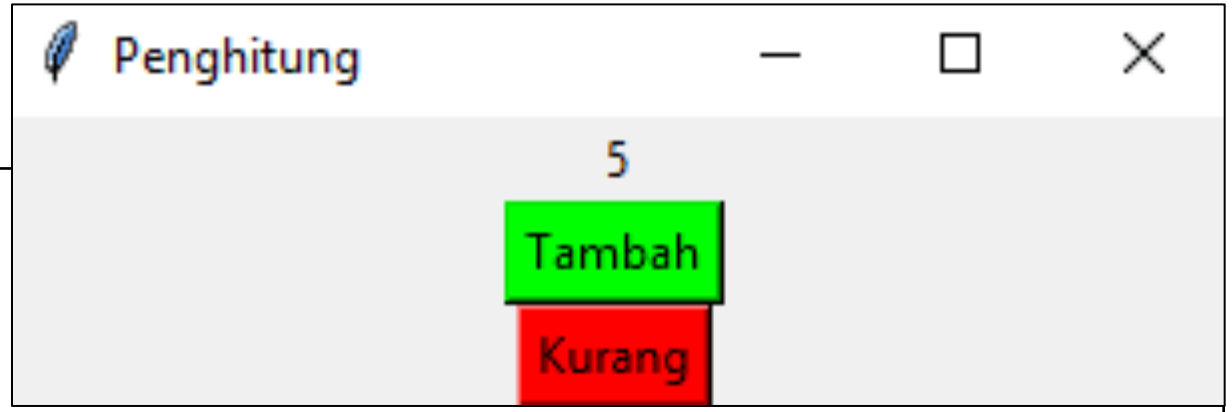


Live Coding : Penghitung



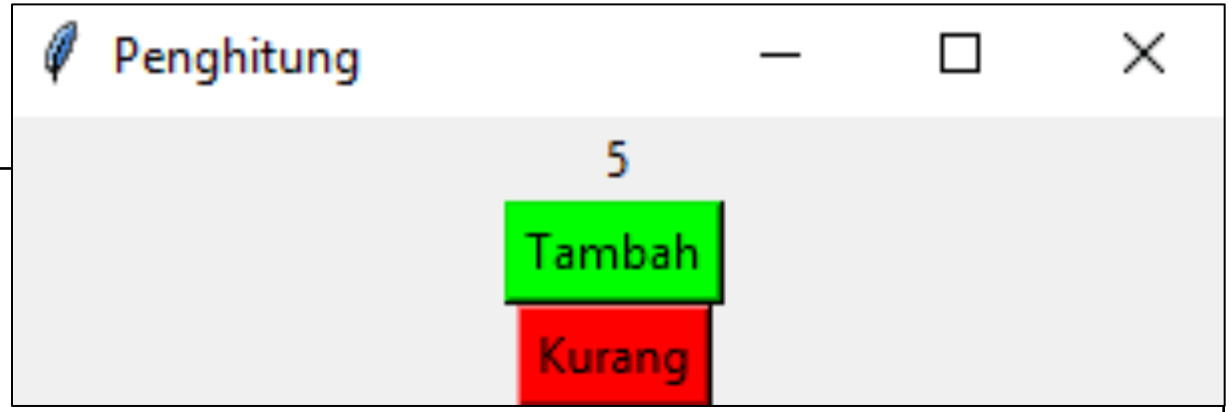

```
from tkinter import *

class Penghitung:

    def __init__(self, master):
        self.master = master
        master.title(_________)
        master.geometry("300x100")

        self.hitungan = 0
        self.label = Label(master, text=_________)
        self.label.pack()

        self.tambah_button = Button(master, text="Tambah", command=_________, bg="#00FF00")
        self.tambah_button.pack()
        self.kurang_button = Button(master, text="Kurang", command=_________, bg="#FF0000")
        self.kurang_button.pack()
```

Live Coding : Penghitung



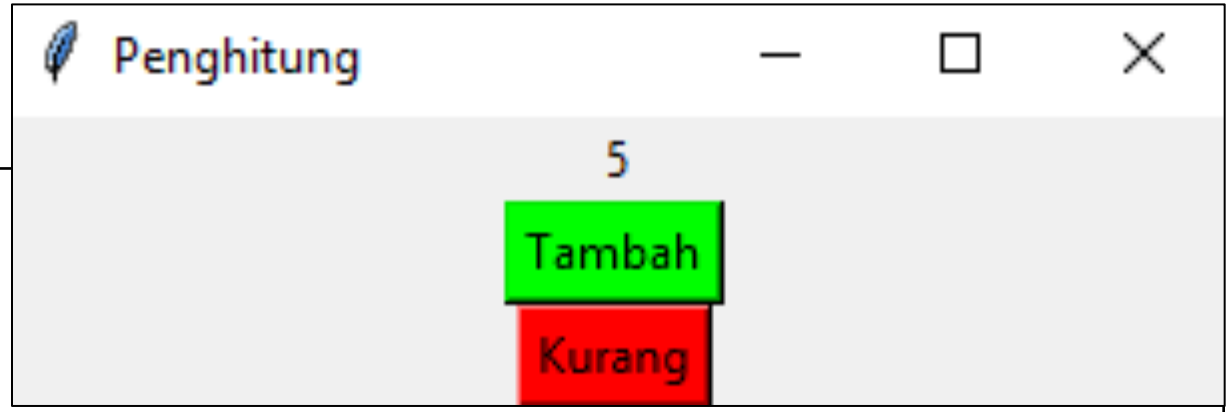

```
# ....

    def __________(self):
        self.hitungan += 1
        self.__________ = self.hitungan

    def __________(self):
        self.hitungan -= 1
        self.__________ = self.hitungan

root = Tk()
penghitung = Penghitung(root)
root.mainloop()
```

Live Coding : Penghitung




```
from tkinter import *

class Penghitung:

    def __init__(self, master):
        self.master = master
        master.title("Penghitung")
        master.geometry("300x100")

        self.hitungan = 0
        self.label = Label(master, text=self.hitungan)
        self.label.pack()

        self.tambah_button = Button(master, text="Tambah", command=self.tambah, bg="#00FF00")
        self.tambah_button.pack()
        self.kurang_button = Button(master, text="Kurang", command=self.kurang, bg="#FF0000")
        self.kurang_button.pack()
```

Live Coding : Penghitung




```
# ....

    def tambah(self):
        self.hitungan += 1
        self.label['text'] = self.hitungan

    def kurang(self):
        self.hitungan -= 1
        self.label['text'] = self.hitungan

root = Tk()
penghitung = Penghitung(root)
root.mainloop()
```

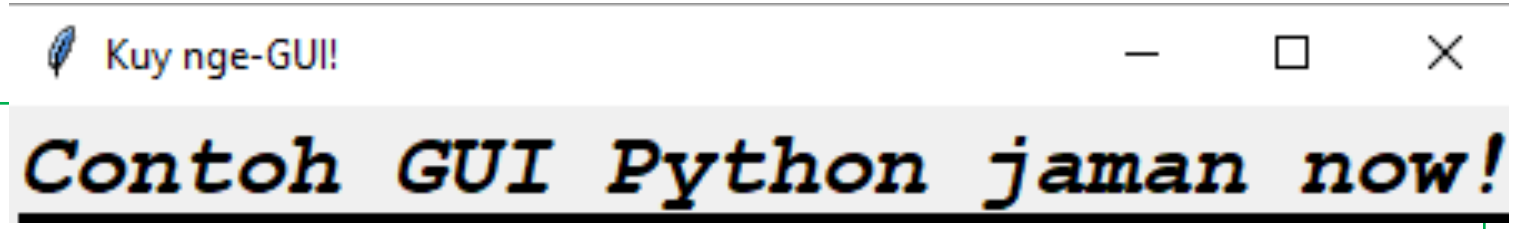
Daftar Atribut Widget

Common Widget Properties	Description
`bg`	Background color.
`fg`	Foreground color.
`width`	Width in pixels
`height`	Height in pixels
`borderwidth`	The size of the border in pixels.
`text`	Text displayed on the widget.
`font`	The font used for text on the widget.
`cursor`	The shape of the cursor when the cursor is over the widget.
`activeforeground`	The color of the text when the widget is activated.
`activebackground`	The color of the background when the widget is activated.
`image`	An image to be displayed on the widget.

Live Coding : Font

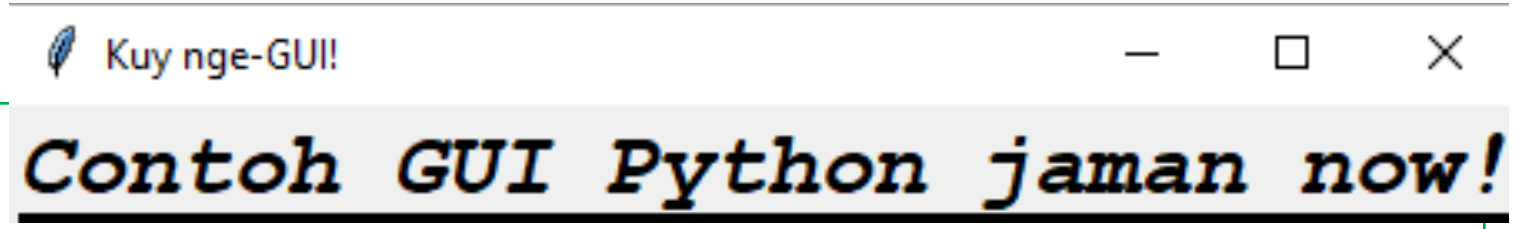
```
from tkinter import *

class MyFirstGUI:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-GUI!")

        self.label = Label(master, text="Contoh GUI Python jaman now!", font="Courier 20 bold
italic underline")
        self.label.pack()

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```

Live Coding : Font




```
from tkinter import *

class MyFirstGUI:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-GUI!")

        self.label = Label(master, text="Contoh GUI Python jaman now!", font="Courier 20 bold
italic underline")
        self.label.pack()

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```

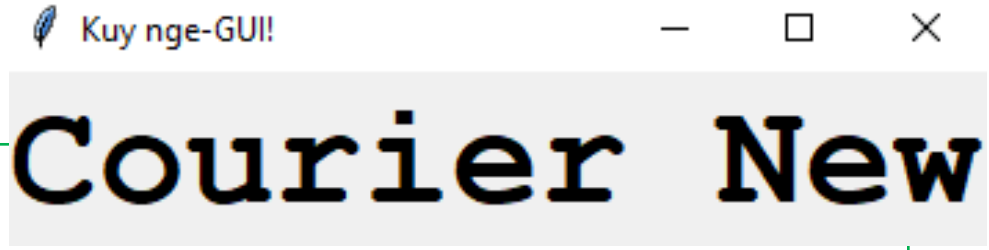
Live Coding : Font




```
from tkinter import *

class MyFirstGUI:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-GUI!")

        self.label = Label(master, text="Contoh GUI Python jaman now!", font="Courier 20 bold
italic underline")
        self.label.pack()

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```

- Times
- Helvetica
- Σψμβολ
- **Courier New**

Live Coding : Font




```
from tkinter import *
from tkinter.font import *

class MyFirstGUI:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-GUI!")

        label_font = Font(family='Courier New', size=40, weight='bold')
        self.label = Label(master, text="Courier New", font=label_font)
        self.label.pack()

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```

Daftar Widget

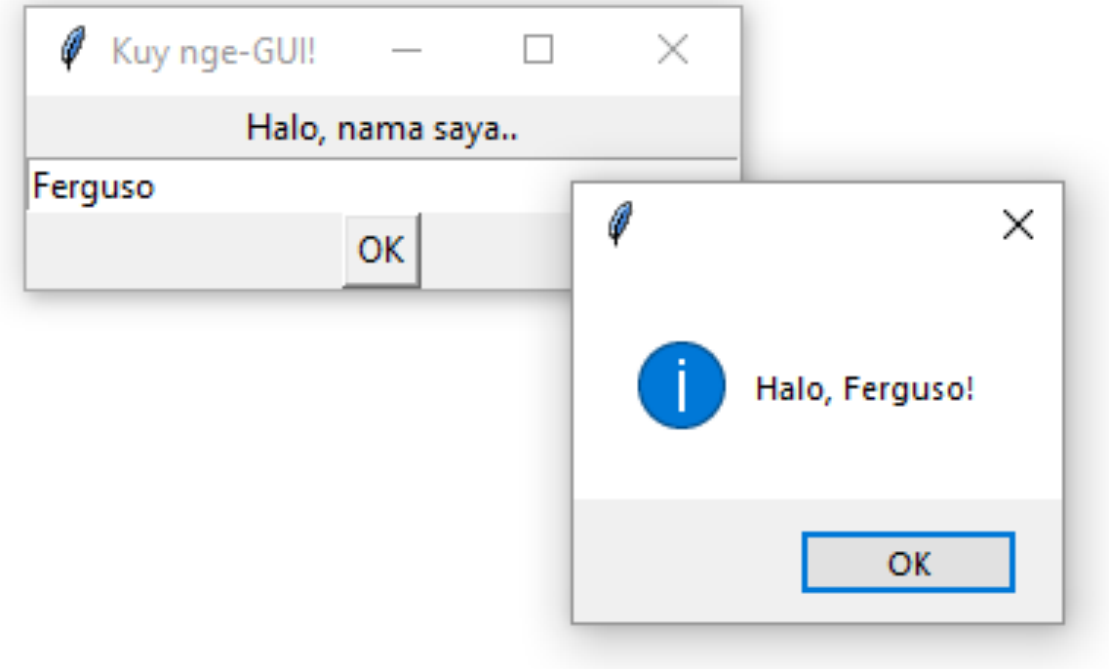
Widget Class	Description
Button	A simple button, used to execute a command.
Canvas	An area to display graphical elements like lines, rectangles or text.
Checkbutton	Clicking a check button toggles between the values.
Entry	A text entry field, also called a text field or a text box.
Frame	A container widget for containing other widgets.
Label	Displays text or an image.
Menu	A menu pane, used to implement pull-down and popup menus.
Menubutton	A menu button, used to implement pull-down menus.
Message	Displays a text. Similar to the label widget, but can automatically wrap text to a given width or aspect ratio.
Radiobutton	Clicking a radio button sets the variable to that value, and clears all other radio buttons associated with the same variable.
Text	Formatted text display. Allows you to display and edit text with various styles and attributes.

Live Coding: Entry Widget

```
from tkinter import *
from tkinter.messagebox import showinfo

class Formulirku:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-GUI!")

        self.label = Label(master, text="Halo, nama saya..")
        self.label.pack()

        self.nama = StringVar()
        self.field_nama = Entry(master, textvariable=self.nama, width=40)
        self.field_nama.pack()

        self.button = Button(master, text="OK", command=self.edit_nama)
        self.button.pack()

    def edit_nama(self):
        showinfo(message="Halo, {}!".format(self.nama.get()))

root = Tk()
my_gui = Formulirku(root)
root.mainloop()
```

Live Coding: Entry Widget

```
from tkinter import *
from tkinter.messagebox import showinfo

class Formulirku:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-GUI!")

        self.label = Label(master, text="Halo, nama saya..")
        self.label.pack()

        self.nama = StringVar()
        self.field_nama = Entry(master, textvariable=self.nama, width=40)
        self.field_nama.pack()

        self.button = Button(master, text="OK", command=self.edit_nama)
        self.button.pack()

    def edit_nama(self):
        showinfo(message="Halo, {}!".format(self.nama.get()))

root = Tk()
my_gui = Formulirku(root)
root.mainloop()
```



Widget Variables

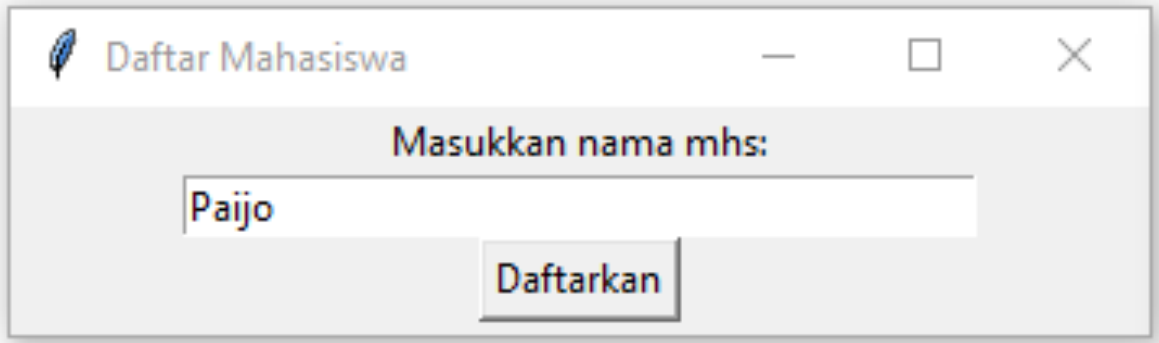
- Beberapa widget memiliki variabel spesial bawaan tkinter
- Nilai dari variabel itu dapat diatur via **.set(new_value)** dan dapat diambil via **.get()**

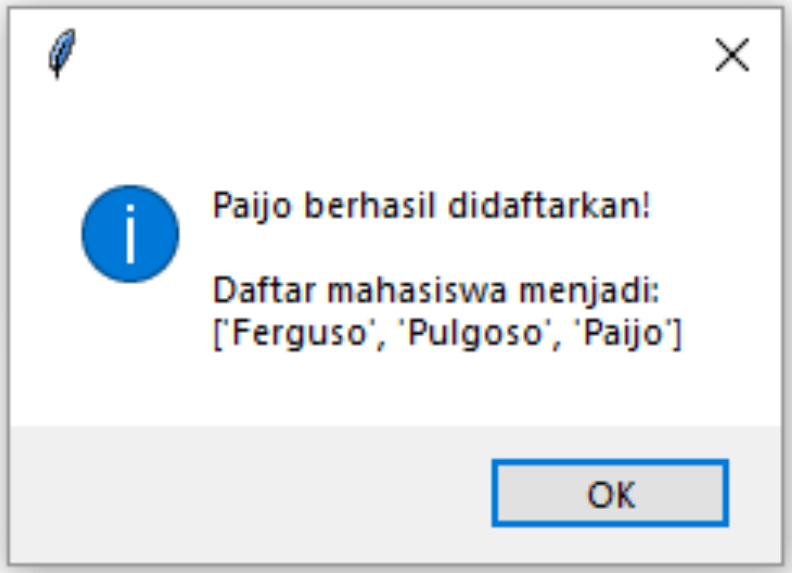
`BooleanVar`	A tk object that holds a single Boolean value
`IntVar`	A tk object that holds a single integer value
`DoubleVar`	A tk object that holds a single double value
`StringVar`	A tk object that holds a single string value

Contoh penggunaan (untuk widget Entry):

```
self.nama = StringVar()
self.field_nama = Entry(master, textvariable=self.nama)
self.field_nama.pack()
```

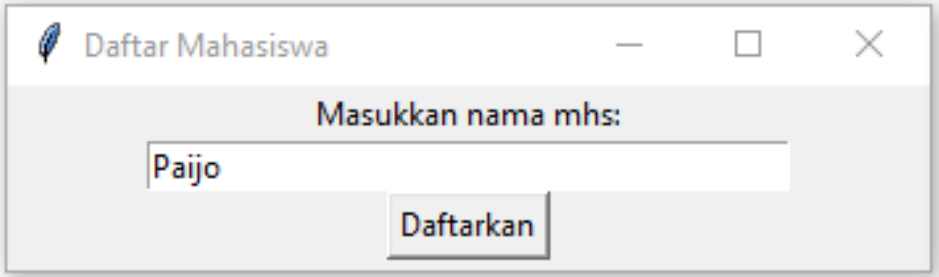
Live Coding : Daftar Mahasiswa

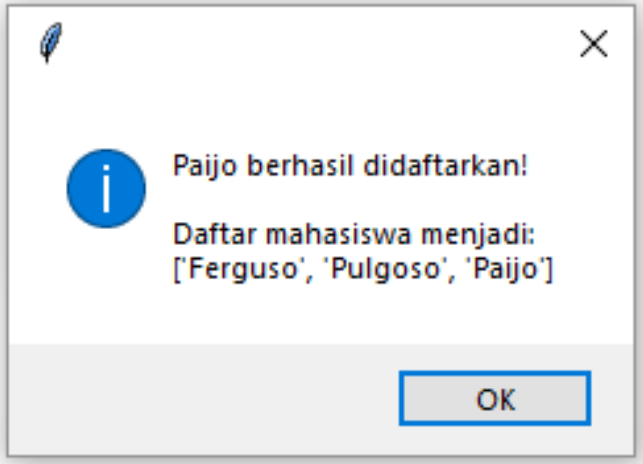




Live Coding : Daftar Mahasiswa

```
from tkinter import *
from tkinter.messagebox import showinfo

class DaftarMhs:

    daftar_mhs = []

    def __init__(self, master):
        self.master = master
        master.title("Daftar Mahasiswa")
        master.geometry("350x70")

        self.label = Label(master, text="Masukk
        self.label.pack()

        self.nama = StringVar()
        self.field_nama = Entry(master, textvariable=self.nama, width=40)
        self.field_nama.pack()

        self.button = Button(master, text="Daftarkan", command=self.daftar)
        self.button.pack()

    def daftar(self):
        mhs = self.nama.get()
        DaftarMhs.daftar_mhs.append(mhs)
        showinfo(message="{} berhasil didaftarkan!\n\nDaftar mahasiswa menjadi:\n{}".format(mhs,DaftarMhs.daftar_mhs))
        self.nama.set("")

root = Tk()
my_gui = DaftarMhs(root)
root.mainloop()
```





Live Coding : Radio Button

```
from tkinter import *

class MyFirstGUI:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-GUI!")

        self.nilai = IntVar()
        self.nilai.set(0) # atur nilai bawaan/default
        rb_pria = Radiobutton(master, text="Pria", variable=self.nilai, value=0, command=self.gender)
        rb_wanita = Radiobutton(master, text="Wanita", variable=self.nilai, value=1, command=self.gender)
        rb_pria.pack()
        rb_wanita.pack()
        print("Nilai:", self.nilai.get())

    def gender(self):
        print("Nilai:", self.nilai.get())

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```

Live Coding : Radio Button

```
...

        self.nilai = IntVar()
        self.nilai.set(0) # atur nilai bawaan/default
        rb_pria = Radiobutton(master, text="Pria", variable=self.nilai, value=0, command=self.gender)
        rb_wanita = Radiobutton(master, text="Wanita", variable=self.nilai, value=1, command=self.gender)
        rb_pria.pack()
        rb_wanita.pack()
        print("Nilai:", self.nilai.get())

    def gender(self):
        print("Nilai:", self.nilai.get())
```

Live Coding : Menampilkan Gambar

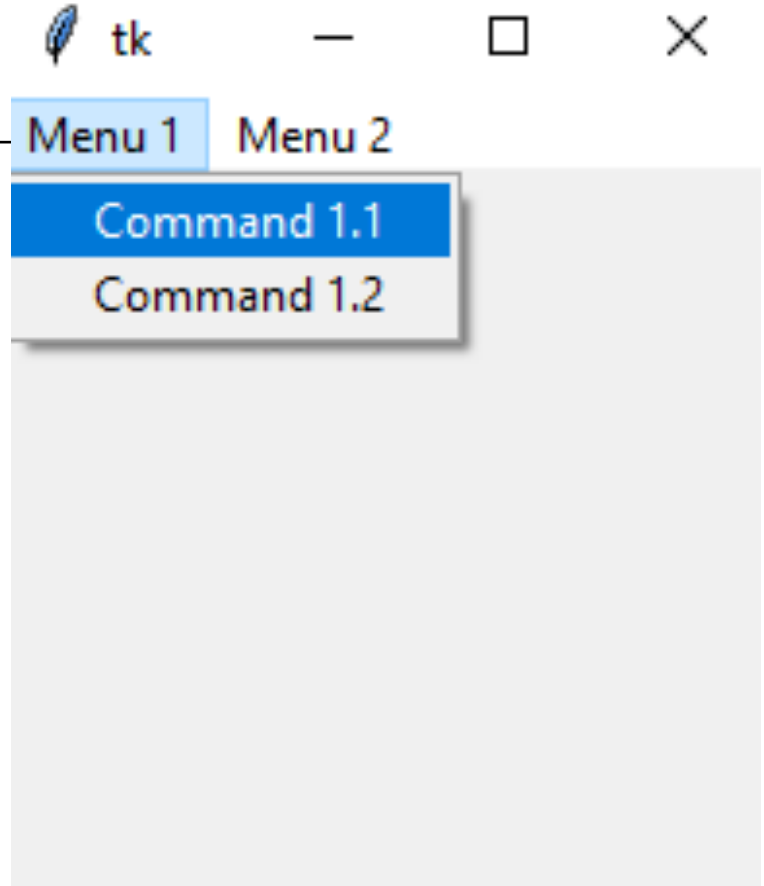
```
from tkinter import *
from PIL import ImageTk, Image

class MyFirstGUI:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-GUI!")
        img = ImageTk.PhotoImage(Image.open('myimage.jpg'))
        self.label = Label(image=img)
        self.label.image = img
        self.label.pack()

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```

Live Coding : Membuat Menu

```
from tkinter import *

class MyFirstGUI:

    def __init__(self, master):
        self.master = master
        menubar = Menu(master)
        master['menu'] = menubar

        block_menu = Menu(menubar, tearoff = 0)
        menubar.add_cascade(label = "Menu 1", menu = block_menu)
        block_menu.add_command(label = "Command 1.1")
        block_menu.add_command(label = "Command 1.2")
        block_menu2 = Menu(menubar, tearoff = 0)
        menubar.add_cascade(label = "Menu 2", menu = block_menu2)
        block_menu2.add_command(label = "Command 2.1")

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```

Live Coding : Membuat Menu

```
from tkinter import *

class MyFirstGUI:

    def __init__(self, master):
        self.master = master
        menubar = Menu(master)
        master['menu'] = menubar

        block_menu = Menu(menubar, tearoff = 0)
        menubar.add_cascade(label = "Menu 1", menu = block_menu)
        block_menu.add_command(label = "Command 1.1")
        block_menu.add_command(label = "Command 1.2")
        block_menu2 = Menu(menubar, tearoff = 0)
        menubar.add_cascade(label = "Menu 2", menu = block_menu2)
        block_menu2.add_command(label = "Command 2.1")

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```



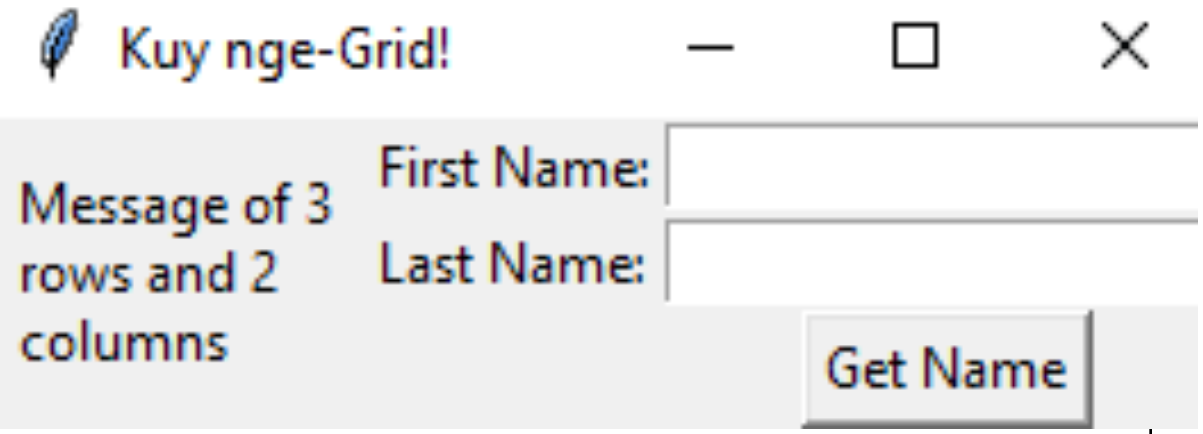
Grid Manager

- Grid manager menggunakan sistem tabel
- Widget dapat ditempatkan pada **cell di baris dan kolom tertentu**
- Bisa digunakan rowspan dan columnspan sehingga widget dapat menempati beberapa baris dan kolom sekaligus

Pesan untuk diingat :

- GUI memudahkan interaksi pengguna
- GUI terdiri atas widgets (button, menu, dsb)
- Widgets tersebut dapat diberi event listener (penanganan event)
- Widgets tersebut juga dapat dimodifikasi (misalkan teksnya, warnanya)
- Geometry/layout manager untuk mengatur penempatan widgets

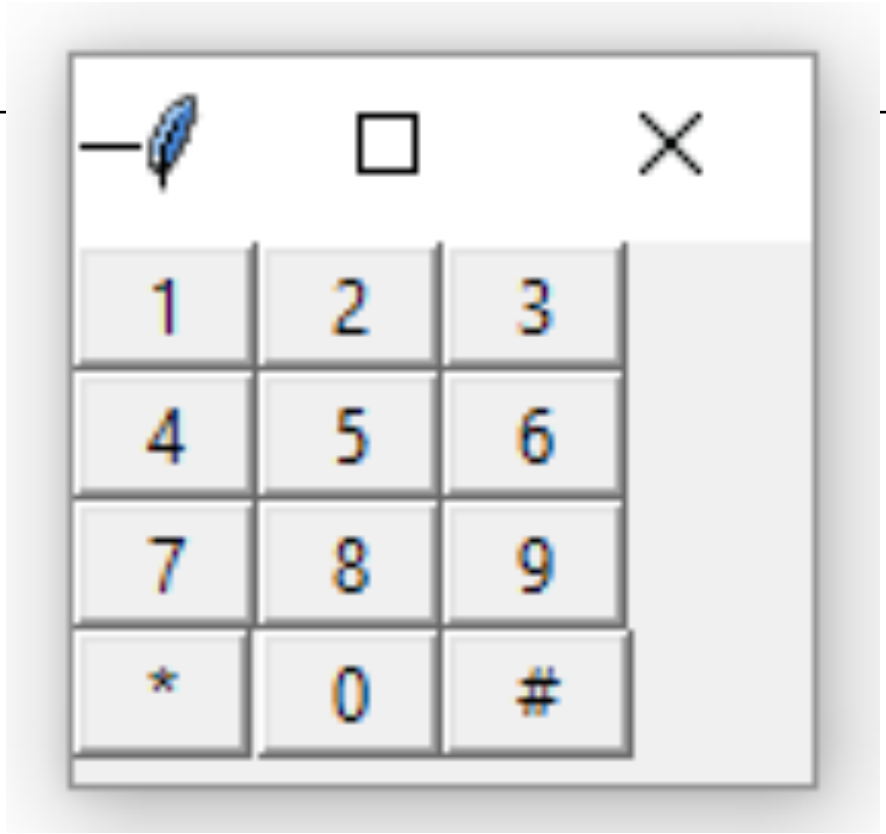
Live Coding : Grid Manager




```
from tkinter import *

class MyGrid:

    def __init__(self, master):
        self.master = master
        master.title("Kuy nge-Grid!")

        message = Message(master, text = "Message of 3 rows and 2 columns")
        message.grid(row = 1, column = 1, rowspan = 3, columnspan = 2)
        Label(master, text = "First Name:").grid(row = 1, column = 3)
        Entry(master).grid(row = 1, column = 4)
        Label(master, text = "Last Name:").grid(row = 2, column = 3)
        Entry(master).grid(row = 2, column = 4)
        Button(master, text = "Get Name").grid(row = 3, column = 4)

root = Tk()
my_grid = MyGrid(root)
root.mainloop()
```

Live Coding : Grid Manager

```
from tkinter import Tk, Label, RAISED

class Keypad:

    def __init__(self, root):
        labels = [['1', '2', '3'],
          ['4', '5', '6'],
          ['7', '8', '9'],
          ['*', '0', '#']]

        for r in range(4):
            for c in range(3):
                # create label for row r and column c
                label = Label(root,
                              relief=RAISED,
                              padx=10,
                              text=labels[r][c])
                # place label in row r and column c
                label.grid(row=r, column=c)

root = Tk()
keypad = Keypad(root)
root.mainloop()
```



```
class ThankYou:

    def __init__(self):
        print("Thank you!")

thanks = ThankYou()
```